

REDCap Survey Sync — Record-ID Keyed by the Study SID

How a submitted patient survey is written into the REDCap record whose **record_id** is the study **SID** — traced step by step through the backend code.

Scope Survey submission → REDCap import **Mode** REDCAP_RECORD_ID_MODE = test

Code routes_surveys.py · redcap_mapping.py · deid.py **Audience** Internal / engineering

1 The operating model — record_id is the study SID

A survey is written to the REDCap record whose `record_id` is the patient's study SID. The **coordinator** names the REDCap record after the study SID (e.g. `SID_22`), and the survey is written straight to that record — no lookup, no mapping table, no extra field.

THIS IS THE PRODUCTION BEHAVIOUR IN USE

Keying the record by SID is the intended production workflow for this project. In the code the behaviour is selected by `REDCAP_RECORD_ID_MODE = test` in `app/Backend/.env` — the value's name is legacy; leave it as-is.

2 Prerequisite — the coordinator creates the record

Before any survey is submitted

In REDCap, the coordinator registers the patient and sets the `record_id` to the study SID (for example `SID_22`). This yields an empty REDCap record keyed by SID. The seed / create scripts in `app/Backend/scripts/` produce the exact same shape when records must be pre-created in bulk.

CONSISTENCY REQUIREMENT

Every patient's `record_id` must follow the same SID form. If some records are auto-numbered (`1`, `2`) and others are `SID_22`, surveys for the auto-numbered patients will not match — the backend looks for the SID string.

3 End-to-end flow of a single submission

1 Frontend posts the survey

`POST /api/surveys/submit → submit_survey() · routes_surveys.py:504`

The request carries a **hashed** `speaker`, not the SID. De-identification keeps the real study SID out of the URL and payload; SID 22 arrives disguised as a numeric token.

2 Persist to PostgreSQL first — before REDCap

`routes_surveys.py:547 · sid = unhash_patient_sid() · deid.py:42`

The answers are saved to Postgres immediately, un-hashing the speaker back to `"SID_22"` (affine-cipher inverse). If REDCap later fails, the response is never lost.

3 Proceed to REDCap only if configured

`routes_surveys.py:566 → import_to_redcap() · routes_surveys.py:363`

Guarded by `if REDCAP_ENABLED:` (true only when both the API URL and token are set).

4 Recover the SID, then resolve it to a record_id

`routes_surveys.py:396–397 · resolve_record_id() · redcap_mapping.py`

The speaker is un-hashed again to `"SID_22"`, then converted to a REDCap `record_id`. **This is the one line that decides the record.** The function returns the SID unchanged, so `record_id = "SID_22"`. If it returns nothing, the submission is left *pending* and is not pushed.

5 Look up the field mapping for the survey type

`routes_surveys.py:405 · FRONTEND_TO_REDCAP_MAPPING[survey_type]`

A dictionary that translates each frontend answer key into its REDCap field name (e.g. DCS items → `dc1_v2 ... dc16_v2`).

6 Convert answers into REDCap fields

`routes_surveys.py:419–427 · transform_value()`

Each answer is transformed to the coded value REDCap expects and collected into `redcap_fields`. Unmapped keys are skipped.

7 Mark the instrument complete

`routes_surveys.py:432–434 · SURVEY_COMPLETE_FIELDS`

The instrument's `_complete` field is set to `"2"` (Complete — the green status in REDCap).

8 Call the record-import function

`routes_surveys.py:465 → import_to_redcap_record() · routes_surveys.py:1560`

Invoked with `record_id="SID_22"`. Free-text surveys use `overwrite`; radio/scale surveys use `normal`.

9 Write to REDCap over HTTP

```
routes_surveys.py:1646 (assemble) · :1712 (POST content=record)
```

The record is assembled as `{"record_id": "SID_22", ...fields}` and POSTed to the REDCap API. The decisive parameter is `forceAutoNumber = false` — REDCap keeps the SID we sent as the `record_id` instead of assigning its own number.

10 Record the sync outcome back in the database

```
routes_surveys.py:568–571
```

`redcap_synced`, `redcap_record_id` (= `"SID_22"`) and any `redcap_error` are written to the log row and committed.

4 The two mechanisms that make it work

MECHANISM A — RESOLVE_RECORD_ID RETURNS THE SID AS-IS

```
# redcap_mapping.py — resolve_record_id()
if get_settings().redcap_record_id_mode != "production":
    return sid          # TEST: record_id == SID → "SID_22"
```

No export, no mapping, no `study_sid` field. The SID string *is* the `record_id`.

MECHANISM B — FORCEAUTONUMBER = FALSE PINS THAT RECORD_ID

```
# routes_surveys.py:1712 — POST to REDCap
data={ 'content': 'record', 'overwriteBehavior': overwrite,
       'forceAutoNumber': 'false', # keep "SID_22" verbatim
       'data': json.dumps([redcap_record]) }
```

Even though the project has auto-numbering enabled, this flag tells REDCap to use the `record_id` we supplied. That is what allows a SID-named record to receive the survey.

5 Flow at a glance

